

SIMATS ENGINEERING

ASSESSMENT TOOL 3 Mock

Interview

COURSE NAME: COMPUTER VISION WITH OPENCV

COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO5: Understand video processing - motion computation - 3D vision - geometry. (BTL 6)

Assessment Tool Weightage: 10%

Mock Interview Questions

OpenCV Basics & Pipeline Thinking

1. Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently. Clearly justify your design choices and discuss potential challenges and optimizations.

ANS:

OpenCV Pipeline for Multiple Image Processing

Objective

The objective is to design an efficient OpenCV pipeline that reads multiple images from a folder, processes each image, displays the result, and optionally saves the processed images.

Complete Pipeline

Input Folder



Get Image Files



Read Image



Check Image Validity



Resize Image



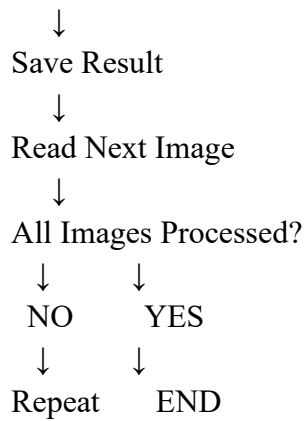
Preprocessing



Image Processing



Display Result



Pseudocode

ALGORITHM MultipleImageProcessing

INPUT:

Folder containing multiple images

OUTPUT:

Processed and displayed images

BEGIN

Import OpenCV

Import operating system library

Set input_folder

Set output_folder

Get all image files from input_folder

FOR each image_file in image_files DO

 Read image using cv2.imread()

 IF image is not loaded THEN

 Display "Invalid or corrupted image"

 CONTINUE

 END IF

 Resize image to a suitable resolution

 Convert image from BGR to grayscale

 Apply Gaussian Blur to remove noise

Apply required image processing
Example: Canny Edge Detection

Display original image

Display processed image

Save processed image to output_folder

Wait for a short time

IF user presses 'q' THEN
 BREAK
END IF

END FOR

Close all OpenCV windows

Display "Processing Completed"

END

Example OpenCV Code

```
import cv2
```

```
import os
```

```
input_folder = "images"
```

```
output_folder = "output"
```

```
os.makedirs(output_folder, exist_ok=True)
```

```
# Get image files
```

```
image_files = [
```

```
    f for f in os.listdir(input_folder)
```

```
    if f.lower().endswith((".jpg", ".jpeg", ".png", ".bmp"))
```

```
]
```

```
for filename in image_files:
```

```
    path = os.path.join(input_folder, filename)
```

```
# Read image
```

```

image = cv2.imread(path)

# Check image
if image is None:
    print("Unable to read:", filename)
    continue

# Resize
image = cv2.resize(image, (640, 480))

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Remove noise
filtered = cv2.GaussianBlur(gray, (5, 5), 0)

# Image processing - Edge Detection
edges = cv2.Canny(filtered, 100, 200)

# Display
cv2.imshow("Original Image", image)
cv2.imshow("Processed Image", edges)

# Save result
output_path = os.path.join(output_folder, filename)
cv2.imwrite(output_path, edges)

# Exit if 'q' is pressed
if cv2.waitKey(100) & 0xFF == ord('q'):
    break

```

```
cv2.destroyAllWindows()
```

Justification of Design Choices

1. Image Acquisition

cv2.imread() is used to read each image from the folder. File extensions are filtered so that only supported image formats are processed.

2. Image Validation

The system checks whether the image was successfully loaded. Corrupted or unsupported images are skipped instead of stopping the complete pipeline.

3. Resizing

Large images are resized to 640×480 . This reduces the number of pixels and therefore decreases processing time and memory usage.

4. Grayscale Conversion

The image is converted from BGR to grayscale because many image processing operations do not require color information. This reduces computational complexity.

5. Noise Removal

Gaussian Blur removes small noise and smooths the image. This improves the reliability of further processing.

6. Image Processing

Canny Edge Detection is used as an example processing operation. It identifies important boundaries in the image.

7. Display

cv2.imshow() displays both the original and processed images. cv2.waitKey() controls the display duration and allows the user to stop processing.

8. Output

cv2.imwrite() saves the processed image in an output folder for later use.

Potential Challenges

1. **Large image sizes:** High-resolution images require more memory and processing time.
2. **Corrupted images:** Some files may not be readable.
3. **Different formats:** The folder may contain unsupported file types.
4. **Large number of images:** Processing thousands of images can take a long time.
5. **Display overhead:** Displaying every image can slow down batch processing.
6. **Memory usage:** Keeping all processed images in memory can cause high RAM usage.

Optimizations

1. Resize large images before processing.
2. Process images one at a time instead of loading all images into memory.
3. Use grayscale when color information is unnecessary.
4. Avoid unnecessary image copies.
5. Reduce or disable display during large batch processing.
6. Use multiprocessing for CPU-intensive operations.
7. Save results directly to the output folder.
8. Skip corrupted or unsupported files.

OpenCV Functions Used

Function	Purpose
cv2.imread()	Reads an image
cv2.resize()	Resizes an image
cv2.cvtColor()	Converts BGR to grayscale
cv2.GaussianBlur()	Removes noise
cv2.Canny()	Detects edges
cv2.imshow()	Displays images
cv2.waitKey()	Controls display and keyboard input
cv2.imwrite()	Saves processed images
cv2.destroyAllWindows()	Closes OpenCV windows

Conclusion

The proposed OpenCV pipeline efficiently processes multiple images by reading them from a folder, validating, resizing, preprocessing, applying image processing, displaying, and saving the results. Processing images sequentially reduces memory consumption, while resizing and grayscale conversion improve efficiency. Corrupted files are skipped, and multiprocessing can be used for further optimization when processing a very large number of images.

2. A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.

ANS:

Debugging a Video File Not Loading Properly in OpenCV

Objective

The objective is to identify why a video file is not loading properly in OpenCV and provide a structured debugging approach. Possible causes include an incorrect file path, unsupported video format or codec, permission issues, corrupted files, or problems with the OpenCV installation.

Common Causes

1. Incorrect or invalid video file path.
2. Video file does not exist.
3. Unsupported video format or codec.
4. Corrupted video file.
5. OpenCV cannot access the required video backend.
6. Insufficient permissions to access the file.
7. Incorrect use of `cv2.VideoCapture()`.
8. Video file has an unusual encoding or container format.

Structured Debugging Approach

Step 1: Verify the File Path

First, check whether the specified video file actually exists.
`import os`

```
video_path = "video.mp4"
```

```
print(os.path.exists(video_path))
```

If the output is `False`, the file path is incorrect.

Solution: Use the correct path or provide the complete path.

Step 2: Open the Video Using OpenCV

Use `cv2.VideoCapture()` to open the video.

```
import cv2
```

```
cap = cv2.VideoCapture("video.mp4")
```

```
if not cap.isOpened():
```

```
    print("Unable to open video")
else:
    print("Video opened successfully")
isOpened() determines whether OpenCV successfully opened the video source.
```

Step 3: Check Frame Reading

Even if the video opens, individual frames may fail to load.

```
ret, frame = cap.read()
```

if not ret:

```
    print("Unable to read video frame")
```

else:

```
    print("Frame read successfully")
```

Here, ret indicates whether a frame was successfully retrieved.

Step 4: Check Video Properties

If the video opens but behaves incorrectly, check its properties.

```
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
```

```
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
```

```
fps = cap.get(cv2.CAP_PROP_FPS)
```

```
frame_count = cap.get(cv2.CAP_PROP_FRAME_COUNT)
```

```
print("Width:", width)
```

```
print("Height:", height)
```

```
print("FPS:", fps)
```

```
print("Frames:", frame_count)
```

If the values are zero or invalid, OpenCV may not be correctly interpreting the video file.

Step 5: Test with a Different Video

Try loading another standard video file.

```
cap = cv2.VideoCapture("test.mp4")
```

If another video works, the original file may be corrupted or encoded using an unsupported codec.

Step 6: Check Video Format and Codec

A file extension such as .mp4 does not guarantee that the codec inside the file is supported by the installed OpenCV environment.

Common codecs include:

- H.264
- H.265
- MPEG-4
- VP8
- VP9

Solution: Convert the video to a commonly supported format such as MP4 with H.264 encoding.

Step 7: Check OpenCV Installation and Backend

Check the installed OpenCV version.

```
print(cv2.__version__)
```

OpenCV may use different video backends depending on the operating system, such as FFMpeg or platform-specific backends.

If required, try specifying a backend:

```
cap = cv2.VideoCapture(  
    "video.mp4",  
    cv2.CAP_FFMPEG  
)
```

This can help when the default backend cannot decode the video correctly.

Step 8: Check File Permissions

The program must have permission to access the video file.

Solution:

- Check file permissions.
 - Move the video to a directory accessible by the application.
 - Avoid protected system directories.
-

Step 9: Check for a Corrupted Video

Try opening the video using a normal media player.

If the video does not play there either, the file may be corrupted.

Solution: Obtain a valid copy or repair/re-encode the video.

Step 10: Test the Complete Video Loop

After confirming that the video opens, test continuous frame processing.

```
import cv2
```

```
cap = cv2.VideoCapture("video.mp4")
```

```
if not cap.isOpened():
```

```
    print("Error: Cannot open video")
```

```
    exit()
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        print("End of video or frame read error")
```

```
        break
```



```
cv2.imshow("Video", frame)
```

```
if cv2.waitKey(25) & 0xFF == ord('q'):  
    break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

If this program works, the problem was likely related to the previous implementation rather than the video itself.

Debugging Flow

START



Check File Path



File Exists?



YES



Open with VideoCapture()



isOpened() = TRUE?



YES



Read Frame



Frame Successfully Read?



YES



Check Video Properties



Test with Another Video



Check Codec / Format



Check OpenCV Backend



Check File Permissions



Process Video



END

Possible Cause and Solution

Problem	Possible Cause	Solution
Video does not open	Wrong file path	Verify path
isOpened() returns False	Unsupported format/codecs	Convert video or use supported backend
read() returns False	Corrupted video or decoding issue	Test/re-encode video
FPS is 0	Backend/codecs issue	Check backend and codecs
No display	Incorrect imshow()/waitKey() usage	Check display loop
Permission error	File access restrictions	Change file permissions/location
Only some videos work	Codecs incompatibility	Convert to H.264/MP4
OpenCV-related error	Installation/backend issue	Check OpenCV version and installation

Conclusion

The video-loading problem should be debugged systematically rather than changing the code randomly. First, verify the file path and existence, then check VideoCapture() and isOpened(). Next, verify frame reading and video properties. If the code is correct, investigate codecs compatibility, OpenCV video backends, file permissions, and possible video corruption. Testing another known-good video helps isolate whether the problem is with the code or the original video file. This structured approach makes debugging faster and more reliable.

3. You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently, including key OpenCV functions and reasoning.

ANS:

Overlaying Text and Shapes on a Live Video Feed

Objective

The objective is to design an OpenCV-based system that captures a live video feed and overlays text and graphical shapes such as rectangles, circles, and lines in real time. The solution should be efficient and maintain a smooth video display with minimum processing delay.

Complete Pipeline

Live Camera



Capture Frame



Preprocess Frame if Required



Draw Shapes



Add Text



Display Frame



Capture Next Frame



Repeat Continuously



Press 'q'



Release Camera



END

Pseudocode

ALGORITHM LiveVideoOverlay

INPUT:

Live camera feed

OUTPUT:

Live video with text and shapes

BEGIN

Import OpenCV

Open camera

cap ← cv2.VideoCapture(0)

IF camera is not opened THEN

Display "Camera Error"

STOP

END IF

WHILE camera is running DO

Capture frame

ret, frame ← cap.read()

```
IF ret = FALSE THEN  
    BREAK  
END IF
```

Flip frame if required

MODULE 1: DRAW RECTANGLE

Draw rectangle around required region

```
cv2.rectangle(  
    frame,  
    top_left,  
    bottom_right,  
    color,  
    thickness  
)
```

MODULE 2: DRAW CIRCLE

Draw circle at required position

```
cv2.circle(  
    frame,  
    center,  
    radius,  
    color,  
    thickness  
)
```

MODULE 3: DRAW LINE

Draw line between two points

```
cv2.line(  
    frame,  
    start_point,  
    end_point,  
    color,  
    thickness  
)
```

MODULE 4: ADD TEXT

Add text label to the frame

```
cv2.putText(  
    frame,  
    "Live Video",  
    position,  
    font,  
    font_size,  
    color,  
    thickness  
)
```

MODULE 5: DISPLAY

Display the modified frame

```
cv2.imshow(  
    "Live Feed",  
    frame  
)
```

Wait for a short time

```
IF user presses 'q' THEN  
    BREAK  
END IF
```

END WHILE

Release camera

```
cap.release()
```

Close all OpenCV windows

```
cv2.destroyAllWindows()
```

END

Example Python Implementation

```
import cv2
```

```
cap = cv2.VideoCapture(0)
```

```
if not cap.isOpened():  
    print("Unable to open camera")
```

```
exit()
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
    # Draw rectangle
```

```
    cv2.rectangle(
```

```
        frame,
```

```
        (100, 80),
```

```
        (400, 300),
```

```
        (0, 255, 0),
```

```
        2
```

```
    )
```

```
    # Draw circle
```

```
    cv2.circle(
```

```
        frame,
```

```
        (500, 200),
```

```
        50,
```

```
        (255, 0, 0),
```

```
        2
```

```
    )
```

```
    # Draw line
```

```
    cv2.line(
```

```
        frame,
```

```
        (50, 400),
```

```
        (600, 400),
```

```
        (0, 0, 255),
```

```
        2
```

```
    )
```

```
    # Add text
```

```
    cv2.putText(
```

```
        frame,
```

```
        "LIVE VIDEO",
```

```
        (100, 50),
```

```
        cv2.FONT_HERSHEY_SIMPLEX,
```

```
        1,
```

```

        (255, 255, 255),
        2
    )

    # Display frame
    cv2.imshow("Live Feed", frame)

    # Press q to exit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

```

Key OpenCV Functions

Function	Purpose
cv2.VideoCapture()	Captures live video
cap.read()	Reads each video frame
cv2.rectangle()	Draws rectangles
cv2.circle()	Draws circles
cv2.line()	Draws lines
cv2.putText()	Adds text
cv2.imshow()	Displays live video
cv2.waitKey()	Handles keyboard input and controls loop timing
cap.release()	Releases the camera
cv2.destroyAllWindows()	Closes OpenCV windows

Efficiency Considerations

1. Process Frames Directly

The overlay operations are performed directly on the captured frame. This avoids unnecessary image copies and reduces memory usage.

2. Use Simple Drawing Operations

Functions such as `rectangle()`, `circle()`, `line()`, and `putText()` are computationally inexpensive, making them suitable for real-time video.

3. Avoid Unnecessary Processing

If the purpose is only to overlay information, there is no need to perform expensive operations such as complex image segmentation or deep-learning inference.

4. Control Resolution

If the camera produces very high-resolution frames, the resolution can be reduced to improve frame rate.

```
frame = cv2.resize(frame, (640, 480))
```

5. Minimize Display Delay

A small delay such as:

`cv2.waitKey(1)`

allows the video to remain responsive while checking for keyboard input.

Possible Applications

The same technique can be used to display:

- Face detection bounding boxes
- Object detection results
- FPS counters
- Warning messages
- Tracking paths
- Motion detection regions
- Camera information
- Measurement lines and points

Handling Real-Time Constraints

Requirement	Solution
Live video	<code>cv2.VideoCapture()</code>
Text overlay	<code>cv2.putText()</code>
Rectangles	<code>cv2.rectangle()</code>
Circles	<code>cv2.circle()</code>
Lines	<code>cv2.line()</code>
Low processing delay	Lightweight drawing operations
Smooth video	Small <code>waitKey()</code> delay
Memory efficiency	Modify the existing frame directly
Safe termination	q key, <code>release()</code> , and <code>destroyAllWindows()</code>

Conclusion

The proposed system continuously captures frames from a live camera and directly overlays text and shapes using OpenCV drawing functions. `cv2.rectangle()`, `cv2.circle()`, and `cv2.line()` are used for graphical elements, while `cv2.putText()` is used for labels and information. Since these operations are lightweight and performed directly on each frame, the system can operate efficiently in real time. The camera and OpenCV windows are properly released when the user presses q.

Code of Conduct:

I Amalraj P (192524089) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Weight age	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated

Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately